COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 29, 2026	**DATE SUBMITTED:**	September 29, 2026
NAME:	Ramos, Samuel Reyes, Fenzie Aaron A. Rodulfa, Jose Martin C. Salcedo, Lance Santiago, Nina Denise M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK NO. 10.1 GRAPH

1. PROCEDURE OUTPUT

1. Define the following: (10)

A. Graph – A graph is a data structure made up of vertices (nodes) and edges. It is used to represent relationships or connections between objects. (Weisstein, n.d.)

B. Vertex – A vertex is a point or element in a graph. It represents an object or location that can be connected to other vertices. (Weisstein, n.d.)

C. Nodes – Nodes are the individual elements or points in a graph. They are also commonly called vertices. (Weisstein, n.d.)

D. Weight – A weight is a numerical value assigned to an edge. It can represent distance, cost, time, or another measurement. (GeeksforGeeks, 2026)

E. Path – A path is a sequence of connected vertices that allows you to travel from one vertex to another. For example, A → C → B is a path. (GeeksforGeeks, 2026)

F. Directed Graph – A directed graph is a graph where the edges have a specific direction, shown by arrows. For example, A → B means the connection goes from A to B. (Weisstein, n.d.)

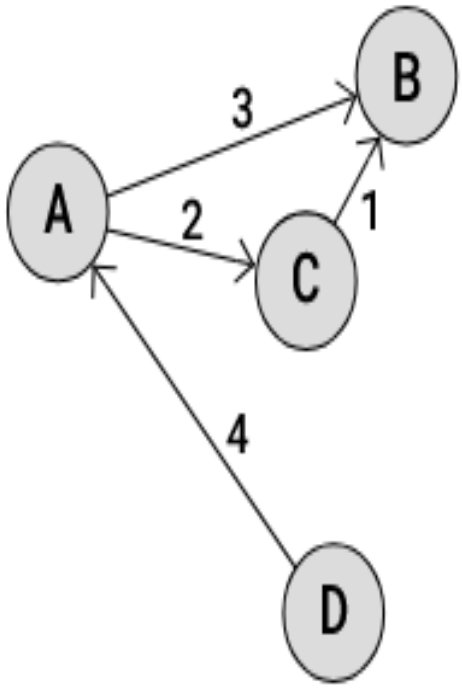
G. Connected Graph – A connected graph is a graph where vertices can be reached from one another through paths. In directed graphs, the direction of the edges must be considered. (GeeksforGeeks, 2026)

H. Directed Cyclic – A directed cyclic graph contains a cycle made of directed edges. A cycle occurs when following the arrows eventually brings you back to the starting vertex. Example: A → B → C → A. (GeeksforGeeks, 2026)

I. Adjacency Matrix – An adjacency matrix is a two-dimensional array used to show the connections between vertices. In a weighted graph, the values can represent the weights of the edges. (Weisstein, n.d.)

J. Adjacency List – An adjacency list represents a graph by listing the vertices directly connected to each vertex. Edge weights can also be included in the list. (Weisstein, n.d.)

2. Identify the parts of the following graph.



Vertex – A, B, C, D
Nodes – A, B, C, D
Weight – 3, 2, 1, 4

Paths:
A → B
A → C
A → C → B
C → B
D → A
D → A → B
D → A →
D → A → C → B

Edges:
(A, B)
(A, C)
(C, B)
(D, A)

3. Plot the adjacency matrix of the graph above.

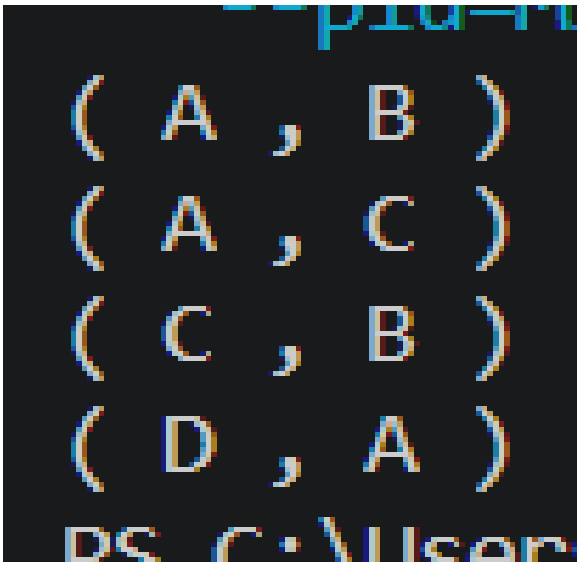
0	3	2	0
0	0	0	0
0	1	0	0
4	0	0	0

4. Create a program in C++ program to create a graph that looks like in the above figure.

Using the same *Graph* class and *Vertex* class from "Module 10.1 Graphs":

```
92  int main() {
93      Graph g;
94
95      // Create vertices A, B, C, and D.
96      g.addVertex('A');
97      g.addVertex('B');
98      g.addVertex('C');
99      g.addVertex('D');
100
101     // Create the directed connections.
102     g.addEdge('A', 'B');
103     g.addEdge('A', 'C');
104     g.addEdge('C', 'B');
105     g.addEdge('D', 'A');
106
107     g.displayGraph();
108
109     return 0;
110 }
```

OUTPUT:



5. Differentiate BFS vs DFS types of search.

Breadth-First Search (BFS) and **Depth-First Search (DFS)** are two ways of visiting the nodes in a graph. BFS checks the nodes **level by level**, starting with the nodes closest to the starting point. It uses a **queue**, where the first node added is the first one processed. On the other hand, DFS goes **as deep as possible** along one path before going back and checking another path. It uses a **stack or recursion**. In simple terms, **BFS explores wide first, while DFS explores deep first**. Both are useful for finding and visiting nodes in a graph, but they follow different ways of exploring it. (GeeksforGeeks, 2025) | To quickly understand the difference between BFS and DFS, the summarized table below shows their main characteristics and how they differ in the way they explore a graph:

BFS	DFS
Breadth-First Search	Depth-First Search
Explores level by level	Explores as deeply as possible
Uses a Queue	Uses a Stack/Recursion
Visits nearby vertices first	Goes deep before backtracking
Useful for shortest paths in unweighted graphs	Useful for exploring graph paths

REFERENCES

GeeksforGeeks. (2026, January 23). *Basic properties of a graph.* https://www.geeksforgeeks.org/dsa/basic-properties-of-a-graph/

GeeksforGeeks. (2025, November 1). *Difference between BFS and DFS.* https://www.geeksforgeeks.org/dsa/difference-between-bfs-and-dfs/

Weisstein, E. W. (n.d.). *Adjacency list*. MathWorld—A Wolfram Web Resource. https://mathworld.wolfram.com/AdjacencyList.html

Weisstein, E. W. (n.d.). *Adjacency matrix*. MathWorld—A Wolfram Web Resource. https://mathworld.wolfram.com/AdjacencyMatrix.html

Weisstein, E. W. (n.d.). *Graph*. MathWorld—A Wolfram Web Resource. https://mathworld.wolfram.com/Graph.html

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

N/A

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: N/A

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: N/A

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: N/A

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.